BLG222E Computer Organization

Project 3

Due Date: 14.05.2026, 23:59

Continue to design a hardwired control unit for the following architecture. Use the structure that you have designed in **Part 5 of Project 1 and Project 2.**

Part 1: INSTRUCTION FORMAT

The instructions are stored in memory in **little-endian order.** Since the RAM in Project 1 has an 8-bit output, **the instruction register cannot be filled in one clock cycle.** You can load MSB and LSB in 2 clock cycles.

- In the first clock cycle (**T=0**), the LSB of the instruction must be loaded from an address A of the memory to the LSB of IR [i.e., IR(7-0)].
- In the second clock cycle (**T=1**), the MSB of the instruction must be loaded from an address A+1 of the memory to the MSB of IR [i.e., IR(15-8)].
- After the second clock cycle (**T=2**), the instruction starts to execute.

The design includes an active-low reset signal. When the reset becomes active, the ARF and RF are placed into clear mode. The actual clearing of the registers is performed synchronously at the next rising edge of the clock. At that clock edge, all registers in ARF and RF are reset to zero.

There are 2 types of instructions as described below.

1- Instructions with address reference have the format shown in Figure 1.
- The **OPCODE** is a **6-bit field.** (Table 1 is given for opcode definition.)
- The **RSEL** is a **2-bit field.** (Table 2 is given for register selection.)
- The **ADDRESS** is an **8-bit field.**

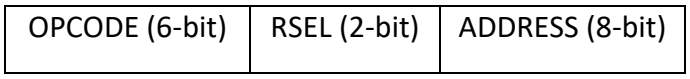

Figure 1: Instructions with an address reference.

2- Instructions without an address reference have the format shown in Figure 2.
- The **OPCODE** is a **6-bit field.** (Table 1 is given for opcode definition.)
- The **DSTREG** is a **3-bit field** that specifies the destination register. (Table 3 is given.)
- The **SREG1** is a **3-bit field** that specifies the first source register. (Table 3 is given.)
- The **SREG2** is a **3-bit field** that specifies the second source register. (Table 3 is given.)
- The least significant bit is unused and have the value 0.

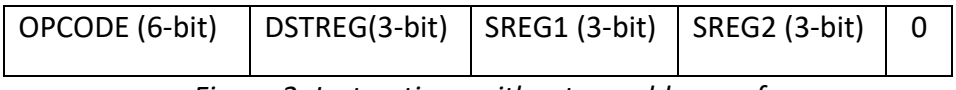

Figure 2: Instructions without an address reference.

Part 2: INSTRUCTION SET

The instruction set of the processor is summarized in Table 3. The instruction set consists of both address-referenced and non-address-referenced instructions, as described in Part 1. Each instruction is defined by its opcode (in hexadecimal format), symbolic name, and a brief description of its functionality. The opcode uniquely identifies the operation to be performed, while the symbol provides a human-readable representation of the instruction. The description column explains the behavior of each instruction, including the operation performed and any changes to registers or memory.

Table 1 defines the register selection encoding for address-referenced instructions. The 2-bit RSEL field is used to select one of the general-purpose registers in Register File (R1–R4). Table 2 defines the register selection encoding for non-address-referenced instructions. The 3-bit fields DSTREG, SREG1, and SREG2 specify the destination register and two source registers, respectively.

Consider the following while implementing the instructions:

- To use the ARF registers for SREG2, you can load them into the scratch registers located in the RF.
- ADDRESS, OFFSET, VALUE, IMMEDIATE are defined in the ADDRESS bits in the instruction.
- The data is stored in memories in little-endian order.
- AR is not automatically incremented after a load/store operation.
- Only operations in the range [0x07,0x15] update the ALU flags.

Table 1: RSel table.

RSEL	REGISTER
00	R1
01	R2
10	R3
11	R4

Table 2: DSTREG/SREG1/SREG2 selection table.

DSTREG/SREG1/SREG2	REGISTER
000	PC
001	PC
010	AR
011	SP
100	R1
101	R2
110	R3
111	R4

Table 3: OPCODE field and symbols for operations and their descriptions.

OPCODE (HEX)	SYMBOL	DESCRIPTION
0x00	BRA	PC ← VALUE
0x01	BNE	IF Z==0 THEN PC ← VALUE
0x02	BEQ	IF Z==1 THEN PC ← VALUE
0x03	BLT	IF N!=O THEN PC ← VALUE
0x04	BGT	IF N==O & Z==0 THEN PC ← VALUE
0x05	BLE	IF N!=O | Z==1 THEN PC ← VALUE
0x06	BGE	IF N==O THEN PC ← VALUE
0x07	INC	DSTREG ← SREG1 + 1
0x08	DEC	DSTREG ← SREG1 – 1
0x09	LSL	DSTREG ← LSL SREG1
0x0A	LSR	DSTREG ← LSR SREG1
0x0B	ASR	DSTREG ← ASR SREG1
0x0C	CSL	DSTREG ← CSL SREG1
0x0D	CSR	DSTREG ← CSR SREG1
0x0E	NOT	DSTREG ← NOT SREG1
0x0F	AND	DSTREG ← SREG1 AND SREG2
0x10	ORR	DSTREG ← SREG1 OR SREG2
0x11	XOR	DSTREG ← SREG1 XOR SREG2
0x12	NAND	DSTREG ← SREG1 NAND SREG2
0x13	ADD	DSTREG ← SREG1 + SREG2
0x14	ADC	DSTREG ← SREG1 + SREG2 + CARRY
0x15	SUB	DSTREG ← SREG1 - SREG2
0x16	MOV	DSTREG ← SREG1
0x17	IMM	Rx ← IMMEDIATE
0x18	POP	SP ← SP + 1, Rx ← M[SP]
0x19	PSH	M[SP] ← Rx, SP ← SP – 1
0x1A	CALL	M[SP] ← PC, SP ← SP – 1, PC ← VALUE
0x1B	RET	SP ← SP + 1, PC ← M[SP]
0x1C	LDR	DSTREG ← M[AR]
0x1D	STR	M[AR] ← SREG1
0x1E	LDA	Rx ← M[ADDRESS]
0x1F	STA	M[ADDRESS] ← Rx
0x20	LDT	Rx ← M[AR+OFFSET]
0x21	STT	M[AR+OFFSET] ← Rx

Part 3: SIMPLE PROGRAM

Implement below program code by determining its binary code. Then write the binary code to ROM. Your final Verilog implementation is expected to fetch the instructions starting from address 0x00, decode them, and execute all instructions one by one.

Since the PC value is initially 0, your code first executes the instruction in memory address 0x00. This instruction is BRA START_ADDRESS where START_ADDRESS is the starting address of your instructions.

```
        BRA   0x8C        # This instruction is written to the memory address 0x00,
                          # The first instruction must be written to address 0x8C
        IMM   R1, 0x06    # R1 is used for iteration number
        IMM   R3, 0xB0
        MOV   AR, R3      # AR is used to track data address: starts from 0xB0
        LDR   R2          # R2 ← M[AR]
        INC   AR, AR      # AR ← AR + 1 (Next Data)
        DEC   R1, R1      # R1 ← R1 – 1 (Decrement Iteration Counter)
LABEL:  LDR   R4          # R4 ← M[AR]
        SUB   R3, R4, R2
        BLE   SKIP        # SKIP if N!=O | Z==1
        MOV   R2, R4
SKIP:   INC   AR, AR      # AR ← AR + 1 (Next Data)
        DEC   R1, R1      # R1 ← R1 – 1 (Decrement Iteration Counter)
        BNE   LABEL       # Go back to LABEL if Z=0 (Iteration Counter > 0)
        STR   R2          # M[AR] ← R2 (Store result at 0xBC)
```

Submission:

Implement your design in Verilog HDL, and upload a single compressed (zip) file to Ninova before the deadline. Only one student from each group should submit the project files (select one member of the group as the group representative for this purpose and note his/her student ID). Example submission file and module name declarations are given as attachments. This compressed file should contain your modules file (.v) for each part, the given simulation file (.v), and a report that contains:

- the number & names of the students in the group
- information about your control unit design
- count of clock cycles each instruction takes
- count of clock cycles given code snippet takes
- explain the purpose of simple program
- task distribution of each group member

Group work is expected for this project. All members of the group must design together. You must ensure that all modules work properly. After designing your project, you can check your results by running the given simulation files. You are expected to get results by running the given .bat file. **The project will be evaluated only using simulation files by running Run.bat file on Vivado 2017.4. There will not be any partial grading for the designs. If your codes get any error, you will not get any grades for that part so make sure that your codes are working with the given simulation files. For detailed information about testing and submission procedures, check the provided *Running Simulation Tests.pdf* file. There will not be any demonstration sessions. You can ask your questions through the Message Board, so do not ask questions through email.**